

Использование языка программирования Julia для решения математических

Часть I

Авторы:

Чуракова Юлия Романовна

*Студентка факультета математики и компьютерных наук
Кубанского Государственного Университета*

Алексеев Евгений Ростиславович

*Кандидат технических наук, доцент
Доцент кафедры информационных технологий
Кубанского Государственного Университета*

Полная версия пособия доступна в репозитории:

<https://github.com/JuliaChurakova/Julia>

Литература

1. Энгхейм, Э. Julia в качестве второго языка / Э. Энгхейм. – Москва: ДМК Пресс, 2023. – 446 с.
2. The Julia Programming Language: сайт. – URL: <https://julialang.org> (дата обращения: 10.10.2025).
3. Белов, Г. В. Краткое описание языка программирования Julia и некоторые примеры его использования/ Г. В. Белов. – Москва: МГТУ им. Н.Э. Баумана, 2024. – 108 с.
4. Шеррингтон, М. Осваиваем язык Julia / М. Шеррингтон. – Москва: ДМК Пресс, 2017. – 416 с.
5. Антонюк, В. А. Язык Julia как инструмент исследователя / В. А. Антонюк. – Москва: Физический факультет МГУ им. М. В. Ломоносова, 2019. – 48 с.

Глава 2 Структуры данных

2.1 Строки в julia

Строка в Julia – это набор символов, заключенных между двойными кавычками " ". Символы вводятся в кодировке UTF-8.

```
In [21]: b = "строка 1\nстрока 2\n"
println(b)
```

```
строка 1
строка 2
```

Функция **length()** – возвращает число символов в строке.

```
In [22]: st = "Конь"
length(st)
```

```
Out[22]: 4
```

Строку можно рассматривать как одномерный массив (вектор). Например, если строка `s="abc"`, то `s[2]='b'`, а `s[end]='c'`. Однако изменять элементы строки присваиванием нельзя, т. е. оператор `s[3]='d'` является ошибочным с точки зрения языка Julia. В этом случае используется функция **replace()**.

```
In [19]: s = "acbccn"
replace(s, 'c'=>'a')
```

```
Out[19]: "aabaan"
```

```
In [21]: s = "acbccn"
s2 = replace(s, 'c' => 'a'; count=1) # Чтобы заменить только первый символ,
```

```
Out[21]: "aabccn"
```

```
In [22]: s = "acbccn"
s2 = replace(s, 'c' => 'a'; count=2) # Чтобы заменить первый и второй символ
```

```
Out[22]: "aabacn"
```

Отметим, что строка и символ – существенно разные понятия языка Julia, поэтому равенство "A" == 'A' является ложным.

Проверить наличие символа s в строке st можно с использованием конструкции **s in st**, которая возвращает true или false

```
In [25]: 'b' in "abc"
```

```
Out[25]: true
```

Проверка того, что подстрока или символ ss входит в строку st осуществляется с использованием функции **occursin(ss,st)**.

```
In [6]: occursin("CCC", "CCCA")
```

```
Out[6]: true
```

findfirst(ss,st) – найти первое вхождение подстроки или символа ss в строке st. Если ss – строка, то результатом является первый и последний индексы подстроки ss в строке st. Если ss – символ, результатом будет индекс, который соответствует номеру символа в строке. Если подстрока или символ не найдены, функция возвращает nothing.

```
In [36]: findfirst("ia", "julia")
```

```
Out[36]: 4:5
```

```
In [37]: findfirst('u', "julia")
```

```
Out[37]: 2
```

findlast(ss,st) – найти последнее вхождение подстроки или символа ss в строке st. Если ss – строка, то результатом является первый и последний индексы подстроки ss в строке st. Если ss – символ, результатом будет индекс, который соответствует номеру символа в строке. Если подстрока или символ не найдены, функция возвращает nothing.

```
In [39]: findlast('m', "comment")
```

Out[39]: 4

Объединение строк производится с использованием символа `*`.

```
In [40]: c="Hello,"  
d="world!"  
e=c*d
```

Out[40]: "Hello,world!"

strip(st) – удаляет пробелы в начале и в конце строки, если строка состоит из пробелов, то strip(st) возвращает пустую строку.

```
In [27]: st = "          Кубанский государственный университет          "  
st = strip(st)
```

Out[27]: "Кубанский государственный университет"

isempty(st) – проверяет, есть ли символы в строке, если нет, возвращает true, иначе – false.

```
In [28]: isempty(st)
```

Out[28]: false

split(st,'R') – разобрать строку на элементы, если в качестве разделителя используется символ R.

```
In [53]: s = "a,bc,d"  
split(s,',')
```

Out[53]: 3-element Vector{SubString{String}}:
"a"
"bc"
"d"

join(A,'R') – преобразовать элементы массива A в строку, используя в качестве разделителя символ R.

```
In [55]: A=[1.0, 2.0, 3.0]  
join(A,',')
```

Out[55]: "1.0,2.0,3.0"

uppercase(st) – преобразовать строку в верхний регистр.

```
In [29]: uppercase(st)
```

Out[29]: "КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ"

lowercase(st) – преобразовать строку в нижний регистр.

```
In [30]: lowercase(st)
```

```
Out[30]: "кубанский государственный университет"
```

titlecase(st) – преобразовать в верхний регистр первый символ каждого слова строки.

```
In [31]: titlecase(st)
```

```
Out[31]: "Кубанский Государственный Университет"
```

string(x) – превратить число x в строку.

```
In [60]: x = 12312424  
string(x)
```

```
Out[60]: "12312424"
```

В Julia используется такое понятие, как **интерполяция**. Его смысл заключается в следующем: если есть строка `st = "123.456"` или число `a = 3.14`, то их значения можно **«встраивать»** (интерполировать) в строку с помощью знака доллара `$`.

```
In [61]: print("1 + 2 = $(1 + 2)")
```

```
1 + 2 = 3
```

```
In [33]: a = 5  
b = 23  
print("a + b = $(a + b)")
```

```
a + b = 28
```

2.2 Массивы в Julia

Массив (англ. array) – это структура данных, которая хранит набор значений, идентифицируемых по индексу или набору индексов.

В Julia индексация массивов начинается с 1.

2.2.1 Способы объявления массива

Существует довольно много способов объявления массива. Рассмотрим некоторые из них.

```
In [26]: # Пустой массив с указанием типа элементов  
Int[]      # Пустой массив целых чисел  
Float64[]  # Пустой массив чисел с плавающей точкой
```

```
String[]      # Пустой массив строк
Any[]         # Пустой массив для элементов любого типа
```

Out[26]: Any[]

```
In [3]: #Одномерный массив с тремя элементами типа Float64, элементы массива не опре
a=Array{Float64,1}(undef,3)
```

Out[3]: 3-element Vector{Float64}:
6.9180327879339e-310
6.9180632143114e-310
6.9180315011767e-310

```
In [4]: #Двумерный массив 3*5 – три строки, пять столбцов типа Float64, элементы мас
a=Array{Float64,2}(undef,3,5)
```

Out[4]: 3×5 Matrix{Float64}:
0.0 1.5e-323 3.0e-323 4.4e-323 4.4e-323
5.0e-324 2.0e-323 3.5e-323 5.0e-323 4.4e-323
1.0e-323 2.5e-323 4.0e-323 5.0e-323 7.4e-323

```
In [5]: #Одномерный массив с десятью элементами типа Float64, элементы массива не ог
a=Vector{Float64}(undef,10)
```

Out[5]: 10-element Vector{Float64}:
3.5e-323
0.0
4.0e-323
0.0
4.0e-323
0.0
1.0e-323
0.0
3.0e-323
0.0

```
In [6]: #Двумерный массив 2*5 – две строки, пять столбцов типа Int64, элементы масси
a=Matrix{Int64}(undef,2,5)
```

Out[6]: 2×5 Matrix{Int64}:
140023162695376 140023126177104 140023179501576 140023126177296 0
140023179501576 140023162689488 140023162689616 0 0

```
In [7]: #При таком объявлении (элементы через пробел) Julia создает двумерный массив
a=[1 2 3 4]
```

Out[7]: 1×4 Matrix{Int64}:
1 2 3 4

```
In [8]: #Создание одномерного массива (вектора)
a=[1, 2, 3, 4]
```

```
Out[8]: 4-element Vector{Int64}:  
 1  
 2  
 3  
 4
```

```
In [9]: a=[1 2; 3 4] #Двумерная матрица
```

```
Out[9]: 2×2 Matrix{Int64}:  
 1 2  
 3 4
```

```
In [12]: a=zeros(3) #Одномерный массив с тремя элементами типа Float64, с нулевыми значениями
```

```
Out[12]: 3-element Vector{Float64}:  
 0.0  
 0.0  
 0.0
```

```
In [11]: a=zeros(3,4) #Двумерный массив 3*4 типа Float64, с нулевыми значениями элементов
```

```
Out[11]: 3×4 Matrix{Float64}:  
 0.0 0.0 0.0 0.0  
 0.0 0.0 0.0 0.0  
 0.0 0.0 0.0 0.0
```

```
In [13]: a=ones(3) #Одномерный массив с тремя элементами типа Float64, с единичными значениями
```

```
Out[13]: 3-element Vector{Float64}:  
 1.0  
 1.0  
 1.0
```

```
In [14]: a=ones(3,4) #Двумерный массив 3*4 типа Float64, с единичными значениями элементов
```

```
Out[14]: 3×4 Matrix{Float64}:  
 1.0 1.0 1.0 1.0  
 1.0 1.0 1.0 1.0  
 1.0 1.0 1.0 1.0
```

```
In [16]: a=Int64.(zeros(3)) #Одномерный массив с тремя элементами типа Int64, с нулевыми значениями
```

```
Out[16]: 3-element Vector{Int64}:  
 0  
 0  
 0
```

```
In [17]: a=zeros(3,4,5) #Двумерный массив 3*4*5 , с нулевыми значениями элементов
```

Out[17]: 3×4×5 Array{Float64, 3}:

```
[:, :, 1] =  
 0.0  0.0  0.0  0.0  
 0.0  0.0  0.0  0.0  
 0.0  0.0  0.0  0.0
```

```
[:, :, 2] =  
 0.0  0.0  0.0  0.0  
 0.0  0.0  0.0  0.0  
 0.0  0.0  0.0  0.0
```

```
[:, :, 3] =  
 0.0  0.0  0.0  0.0  
 0.0  0.0  0.0  0.0  
 0.0  0.0  0.0  0.0
```

```
[:, :, 4] =  
 0.0  0.0  0.0  0.0  
 0.0  0.0  0.0  0.0  
 0.0  0.0  0.0  0.0
```

```
[:, :, 5] =  
 0.0  0.0  0.0  0.0  
 0.0  0.0  0.0  0.0  
 0.0  0.0  0.0  0.0
```

In [19]: `a=fill(5,3,3)` *#Двумерный массив 3*3 типа Int64*

Out[19]: 3×3 Matrix{Int64}:

```
 5  5  5  
 5  5  5  
 5  5  5
```

In [20]: `a=rand(5)` *#Одномерный массив из 5 элементов, заполненный случайными числами*

Out[20]: 5-element Vector{Float64}:

```
0.3084198317399174  
0.29333039483790624  
0.05875249707582253  
0.9398077001166478  
0.6792000600065631
```

In [21]: `a=rand(1:5,5)` *#Одномерный массив из 5 элементов, заполненный случайными целыми*

Out[21]: 5-element Vector{Int64}:

```
5  
1  
1  
1  
2
```

In [24]: `a = 4`
`b = 2`
`c = 20`
`x=collect(a:b:c)` *#Создать одномерный массив, первый элемент которого равен a*

```
Out[24]: 9-element Vector{Int64}:  
         4  
         6  
         8  
        10  
        12  
        14  
        16  
        18  
        20
```

2.2.2 Работа с элементами массива

push!(A,V) – добавить элемент V в конец массива A.

В Julia функция с «!» в конце (например, push!) изменяет исходные данные, поступающие в функцию, а без «!» (например, sort) оставляет их неизменными.

```
In [8]: a = [3, 56, 75, 3]  
        push!(a,23)
```

```
Out[8]: 5-element Vector{Int64}:  
         3  
        56  
        75  
         3  
        23
```

pushfirst!(A,V) – добавить элемент V в начало массива A.

```
In [9]: pushfirst!(a,3455)
```

```
Out[9]: 6-element Vector{Int64}:  
       3455  
         3  
        56  
        75  
         3  
        23
```

insert!(A,N,X) – для массива A типа Vector определена операция вставки элемента X в произвольную позицию N.

```
In [10]: insert!(a,3,10000)
```

```
Out[10]: 7-element Vector{Int64}:
          3455
           3
        10000
           56
           75
           3
           23
```

pop!(a) – удаление последнего элемента массива.

```
In [11]: println(a)
          pop!(a)
          println(a)
```

```
[3455, 3, 10000, 56, 75, 3, 23]
[3455, 3, 10000, 56, 75, 3]
```

popfirst!(a) – удаление первого элемента массива.

```
In [12]: popfirst!(a)
          print(a)
```

```
[3, 10000, 56, 75, 3]
```

deleteat!(a, n) – удаление элемента массива в позиции n.

```
In [13]: deleteat!(a, 2:3)
```

```
Out[13]: 3-element Vector{Int64}:
          3
          75
          3
```

a=a[end:-1:1] – поменять порядок элементов массива на обратный.

```
In [43]: a=a[end:-1:1]
```

```
Out[43]: 3-element Vector{Int64}:
          75
          56
          3
```

length(a) – определить длину массива.

```
In [45]: length(a)
```

```
Out[45]: 3
```

maximum(a) – найти максимальное число массива.

```
In [46]: maximum(a)
```

```
Out[46]: 75
```

minimum(a) – найти минимальное число массива.

```
In [48]: minimum(a)
```

```
Out[48]: 3
```

2.2.3 Выборки

A[i₁:i₂:i₃] – выбрать элементы вектора A с i_1 по i_3 с шагом i_2 .

```
In [49]: a = [3, 4, 5, 0]
```

```
Out[49]: 4-element Vector{Int64}:  
 3  
 4  
 5  
 0
```

```
In [50]: a[2:1:3]
```

```
Out[50]: 2-element Vector{Int64}:  
 4  
 5
```

```
In [51]: b = [3 4 6; 4 9 2; 4 1 10]
```

```
Out[51]: 3×3 Matrix{Int64}:  
 3  4  6  
 4  9  2  
 4  1 10
```

```
In [52]: b[2:3,2:3] #Выбрать элементы матрицы из строк 2, 3 и столбцов 2, 3.
```

```
Out[52]: 2×2 Matrix{Int64}:  
 9  2  
 1 10
```

```
In [54]: b[1:2,:] #Выбрать строки 1 и 2.  
#Двоеточие на месте одного из индексов означает, что выбираем каждый элемент
```

```
Out[54]: 2×3 Matrix{Int64}:  
 3  4  6  
 4  9  2
```

Применительно к элементам массива можно использовать операции с точкой

```
In [55]: a.^2 # вычислить квадрат всех элементов массива
```

```
Out[55]: 4-element Vector{Int64}:
          9
          16
          25
          0
```

```
In [56]: a = [2, 4, 6]
          b = [4, 10, 12]
          a.+b
```

```
Out[56]: 3-element Vector{Int64}:
          6
          14
          18
```

```
In [57]: a.*b
```

```
Out[57]: 3-element Vector{Int64}:
          8
          40
          72
```

```
In [58]: log.(a) #Вычислить натуральный логарифм всех элементов массива
```

```
Out[58]: 3-element Vector{Float64}:
          0.6931471805599453
          1.3862943611198906
          1.791759469228055
```

```
In [59]: 2 .*a
```

```
Out[59]: 3-element Vector{Int64}:
          4
          8
          12
```

2.2.4 Объединение двух массивов

Если массивы **a** и **b** имеют одинаковое число столбцов, то объединить их (по вертикали) можно командой **vcat(a,b)** или **[a;b]**.

```
In [60]: a = [2.34, 4.355, 6.87]
          b = [4, 10, 12]
          vcat(a,b)
```

```
Out[60]: 6-element Vector{Float64}:
          2.34
          4.355
          6.87
          4.0
          10.0
          12.0
```

```
In [61]: [a;b]
```

```
Out[61]: 6-element Vector{Float64}:
 2.34
 4.355
 6.87
 4.0
10.0
12.0
```

Если массивы `a` и `b` имеют одинаковое число строк, то объединить их (по горизонтали) можно командой **`hcat(a,b)`** или **`[a b]`**.

```
In [62]: hcat(a,b)
```

```
Out[62]: 3×2 Matrix{Float64}:
 2.34  4.0
 4.355 10.0
 6.87 12.0
```

```
In [63]: [a b]
```

```
Out[63]: 3×2 Matrix{Float64}:
 2.34  4.0
 4.355 10.0
 6.87 12.0
```

2.2.5 Изменение размерности массива

При необходимости можно изменить размерность массива с использованием функции **`reshape()`**. Допустим, нужно преобразовать одномерный массив `a=[1,2,3,4,5,6]` в двумерный с двумя строками и тремя столбцами. Это можно сделать следующим образом:

```
In [64]: a=[1,2,3,4,5,6]
a=reshape(a,2,3)
```

```
Out[64]: 2×3 Matrix{Int64}:
 1  3  5
 2  4  6
```

```
In [65]: a=reshape(a,6)
```

```
Out[65]: 6-element Vector{Int64}:
 1
 2
 3
 4
 5
 6
```

2.3 Словари

Словари – это неупорядоченные коллекции пар "ключ-значение". Они позволяют эффективно хранить и извлекать данные по ключу. Ключи в словарях должны быть уникальными и неизменяемыми, а значения могут быть любыми и изменяемыми.

2.3.1 Создание словарей

Словари создаются с использованием конструктора **Dict()**.

```
In [66]: # Пустой словарь
d = Dict()

# Словарь с элементами
a = Dict("apple" => 1.2, "banana" => 0.8, "cherry" => 2.5)

Out[66]: Dict{String, Float64} with 3 entries:
  "cherry" => 2.5
  "banana" => 0.8
  "apple"  => 1.2
```

2.3.2 Доступ к элементам словаря

Доступ к значениям словаря осуществляется с помощью ключей. Если ключ не существует в словаре, возникает ошибка `KeyError`.

```
In [67]: a = Dict("milk" => 1.5, "bread" => 2.0)

# Доступ к значениям
b = a["milk"]
print(b)
# Попытка доступа к несуществующему ключу
c = a["butter"] # KeyError
```

1.5

`KeyError: key "butter" not found`

Stacktrace:

```
[1] getindex(h::Dict{String, Float64}, key::String)
   @ Base ./dict.jl:477
[2] top-level scope
   @ In[67]:7
```

2.3.3 Добавление и изменение элементов

Для добавления новых пар "ключ-значение" и изменения существующих используется синтаксис с квадратными скобками.

```
In [68]: # Создание словаря
a = Dict("name" => "Alice", "age" => 30)
println(a)
# Добавление новой пары
```

```
a["city"] = "New York"
println(a)
# Изменение значения существующего ключа
a["age"] = 31
println(a)
```

```
Dict{String, Any}{"name" => "Alice", "age" => 30}
Dict{String, Any}{"name" => "Alice", "city" => "New York", "age" => 30}
Dict{String, Any}{"name" => "Alice", "city" => "New York", "age" => 31}
```

2.3.4 Удаление элементов

Элементы можно удалять с помощью функции **delete!()**.

```
In [69]: # Создание словаря
a = Dict{"name" => "Alice", "age" => 30, "city" => "New York"}
println(a)
# Удаление элемента
delete!(a, "city")
println(a)
```

```
Dict{String, Any}{"name" => "Alice", "city" => "New York", "age" => 30}
Dict{String, Any}{"name" => "Alice", "age" => 30}
```

2.3.5 Проверка наличия ключа

Для проверки наличия ключа в словаре используется функция **haskey()**.

```
In [70]: a = Dict{"milk" => 1.5, "bread" => 2.0}

# Проверка наличия ключа
println(haskey(a, "milk"))
println(haskey(a, "butter"))
```

```
true
false
```

2.3.5 Объединение словарей

Для объединения словарей можно использовать функцию **merge()**.

```
In [74]: dict1 = Dict{"a" => 1, "b" => 2}
dict2 = Dict{"c" => 3, "d" => 4}
dict3 = Dict{"e" => 5, "f" => 6}

# Объединение словарей
dict4 = merge(dict1, dict2, dict3)
```

```
Out[74]: Dict{String, Int64} with 6 entries:
  "f" => 6
  "c" => 3
  "e" => 5
  "b" => 2
  "a" => 1
  "d" => 4
```

2.4 Кортежи

Кортежи (tuples) – это упорядоченные, неизменяемые коллекции элементов различных типов.

2.4.1 Создание кортежей

Кортежи создаются с использованием круглых скобок и запятых для разделения элементов.

```
In [75]: t = (1, "hello", 3.5)
```

```
Out[75]: (1, "hello", 3.5)
```

Кортежи с одним элементом создаются с запятой, чтобы избежать двусмысленности.

```
In [76]: t = (1,)
```

```
Out[76]: (1,)
```

2.4.2 Доступ к элементам кортежа

Элементы кортежа можно получить с помощью индексации.

```
In [77]: t = (10, 20, 30, 40)

# Доступ к первому элементу
first_element = t[1]
println(first_element)
# Доступ к последнему элементу
last_element = t[end]
println(last_element)
```

```
10
40
```

2.4.3 Распаковка кортежей

Распаковка позволяет присвоить элементы кортежа переменным.

```
In [78]: coordinates = (3, 5)

# Распаковка кортежа
x, y = coordinates

println("x = $x, y = $y")
```

x = 3, y = 5

2.4.4 Неизменяемость кортежей

Попытка изменить элемент кортежа приведет к ошибке.

```
In [79]: t = (1, 2, 3)
t[1] = 10
```

MethodError: no method matching setindex! (::Tuple{Int64, Int64, Int64}, ::Int64, ::Int64)
The function `setindex!` exists, but no method is defined for this combination of argument types.

Stacktrace:

```
[1] top-level scope
@ In[79]:2
```

2.4.5 Вложенные кортежи

Кортежи могут содержать другие кортежи, создавая вложенные структуры.

```
In [80]: t = (1, (2, 3), (4, (5, 6)))

# Доступ к элементам вложенного кортежа
a = t[2]
println(a)
b = t[3][2][1]
println(b)
```

(2, 3)
5

2.4.6 Преобразование кортежей

Хотя кортежи неизменяемы, можно создать новый кортеж на основе существующего, добавив или изменив элементы.

```
In [81]: t = (1, 2, 3)

# Добавление элемента
new_t = (t..., 4)
println(new_t)
# Изменение элемента (создание нового кортежа)
```

```
modified_t = (t[1], 42, t[3])  
println(modified_t)
```

(1, 2, 3, 4)

(1, 42, 3)